

Exercício Prático: API de Reservas e Avaliações do Imóvel Lions

Turma: LionsDev

Tópicos: APIs REST com Express.js, Conexão com MongoDB (Mongoose), Múltiplos Schemas, Validações de Schema, CRUD assíncrono com `async/await`, Query Params, Regras de Negócio em JavaScript e Variáveis de Ambiente.

1. Contexto

A startup **Imóvel Lions** precisa de um sistema para gerenciar imóveis de aluguel por temporada, as reservas dos clientes e a experiência dos usuários por meio de avaliações (reviews).

Para este projeto, você precisará modelar três recursos no MongoDB: o **Imóvel**, a **Reserva** e a **Avaliação**. A estrutura deve seguir o modelo usado em aula: `server.js`, `db.js` e `models/`, com as rotas escritas diretamente no `server.js`.

2. Configuração Inicial e Estrutura de Pastas

Organize sua aplicação com a seguinte estrutura de arquivos:

```
Imovellions/
|-- src/
|   |-- models/
|       |-- imovel.js
|       |-- reserva.js
|       `-- avaliacao.js
|   |-- db.js
|   `-- server.js
|-- .env
`-- package.json
```

Configure o arquivo `.env` com a sua URI de conexão do MongoDB Atlas (`MONGO_URI`) e a porta do servidor (`PORT=3000`).

Importante: Neste exercício, deixe todos os endpoints dentro do arquivo `src/server.js`, como foi feito em sala.

2.1 Modelos de Dados (Schemas)

Defina os Schemas do Mongoose detalhados a seguir:

Modelo: Imóvel (`models/imovel.js`)

- `titulo`: Tipo `String`, obrigatório.
- `descricao`: Tipo `String`, obrigatório.
- `localizacao`: Tipo `String`, obrigatório (ex: `"Ubatuba - SP"`).
- `precoNoite`: Tipo `Number`, obrigatório (valor base da diária).
- `capacidadeMaxima`: Tipo `Number`, obrigatório.
- `disponivel`: Tipo `Boolean`, com valor padrão `true`.

Modelo: Reserva (`models/reserva.js`)

- `imovelId`: Tipo `String`, obrigatório. Esse campo deve guardar o `_id` do imóvel reservado.
- `nomeHospede`: Tipo `String`, obrigatório.
- `emailHospede`: Tipo `String`, obrigatório.
- `dataEntrada` (Check-in): Tipo `String`, obrigatória (ex: `"2026-06-15"`).
- `quantidadeNoites`: Tipo `Number`, obrigatória.
- `hospedes`: Array de Objetos, obrigatório, onde cada objeto pode conter:
 - `nome`: Tipo `String`
 - `idade`: Tipo `Number`
- `valorTotal`: Tipo `Number` (calculado automaticamente pela API).
- `status`: Tipo `String`, padrão `"Pendente"` (deve aceitar apenas: `Pendente`, `Confirmada` ou `Cancelada`).

Modelo: Avaliação (`models/avaliacao.js`)

- `imovelId`: Tipo `String`, obrigatório. Esse campo deve guardar o `_id` do imóvel avaliado.
- `nomeUsuario`: Tipo `String`, obrigatório.
- `nota`: Tipo `Number`, obrigatório (deve aceitar apenas valores de `1` a `5`).
- `comentario`: Tipo `String`, obrigatório (mínimo de 10 caracteres).

3. Requisitos Obrigatórios e Regras de Negócio

Você deverá implementar endpoints para gerenciar Imóveis (`/imoveis`), Reservas (`/reservas`) e Avaliações (`/avaliacoes`).

3.1 CRUD de Imóveis

- **Cadastrar Imóvel (`POST /imoveis`)**: Cria um novo imóvel. Retorne status `201`.
- **Listar Imóveis (`GET /imoveis`)**: Retorna todos os imóveis cadastrados.
- **Buscar Imóveis por Localização (`GET /imoveis/busca`)**:

Deve aceitar o Query Param `localizacao`. Retorna apenas os imóveis que contêm o termo pesquisado na localização (ex: buscar por `"uba"` deve retornar imóveis em `"Ubatuba - SP"`).

3.2 Reservas

- **Criar Reserva (`POST /reservas`)**:

Recebe no corpo: `imovelId`, `nomeHospede`, `emailHospede`, `dataEntrada`, `quantidadeNoites` e `hospedes` (array).

Regras de Negócio (Lógica em JavaScript):

1. **Validação de Existência**: A API deve verificar se o imóvel informado realmente existe no banco de dados. Se não existir, retorne status `404` com erro.
 2. **Validação de Capacidade**: Verifique se a quantidade total de hóspedes (tamanho do array `hospedes`) não ultrapassa a `capacidadeMaxima` do imóvel selecionado. Se ultrapassar, retorne status `400` com erro.
 3. **Validação de Noites**: Verifique se `quantidadeNoites` é maior que zero. Se não for, retorne status `400` com erro.
 4. **Cálculo Simples do Valor Total**: Multiplique `quantidadeNoites` pelo `precoNoite` do imóvel.
 5. Se todas as regras passarem, salve a reserva no banco de dados com o `valorTotal` calculado e retorne o documento com status `201`.
- **Listar Reservas (`GET /reservas`)**: Retorna todas as reservas cadastradas.

- **Alterar Status da Reserva (PATCH /reservas/:id/status)**: Permite atualizar apenas o campo `status` da reserva. Retorne status `200`.

3.3 Avaliações

- **Criar Avaliação (POST /avaliacoes)**:

Recebe no corpo: `imovelId`, `nomeUsuario`, `nota` (de 1 a 5) e `comentario`.

Regras de Negócio:

1. **Validação de Imóvel**: Verifique se o imóvel avaliado realmente existe no banco de dados. Caso contrário, retorne status `404`.
 2. Se o imóvel existir, salve a avaliação e retorne o documento criado com status `201`.
- **Listar Avaliações de um Imóvel (GET /avaliacoes/imovel/:imovelId)**:
 - Retorna todas as avaliações feitas para o imóvel especificado.
 - **Regra de Negócio (Média Geral)**: A resposta deve retornar a lista de avaliações e a média aritmética das notas (calculada em JavaScript).
 - **Excluir Avaliação (DELETE /avaliacoes/:id)**:
 - **Regra de Segurança**: O usuário deve enviar seu `nomeUsuario` no corpo da requisição. O backend deve verificar se a avaliação de fato pertence a esse usuário. Se pertencer, remova a avaliação do banco; caso contrário, retorne status `403` (Proibido) com uma mensagem de erro.

4. Desafios Bônus

Depois que a versão obrigatória estiver funcionando, tente adicionar uma ou mais melhorias:

- Usar `mongoose.Schema.Types.ObjectId`, `ref` e `.populate()` para trazer os dados completos do imóvel ao listar reservas.
- Cobrar uma taxa fixa de limpeza de R\$ 80 em cada reserva.
- Cobrar taxa extra por hóspede adicional a partir do 3º hóspede.
- Aplicar desconto de 10% para estadias com 5 noites ou mais.
- Aplicar cupom `"LIONS10"` com mais 10% de desconto.
- Permitir avaliação apenas de usuários que tenham reserva `"Confirmada"` para aquele imóvel.

- Criar **GET /reservas/relatorio** com faturamento total, total de noites e média de hóspedes.

5. Testes Esperados

Realize testes na sua API simulando o seguinte fluxo de validação:

1. Cadastre um imóvel com capacidade máxima de 4 pessoas e **precoNoite** de R\$ 100.
2. Liste todos os imóveis.
3. Busque imóveis por localização usando **GET /imoveis/busca?localizacao=uba**.
4. Crie uma reserva para esse imóvel e verifique se o **valorTotal** foi calculado.
5. Tente criar uma reserva com mais hóspedes do que a capacidade máxima (deve falhar com erro 400).
6. Mude o status da reserva para **"Confirmada"**.
7. Crie uma avaliação para o imóvel.
8. Liste as avaliações do imóvel e verifique se a média foi calculada corretamente.
9. Tente excluir a avaliação enviando um nome de usuário diferente no corpo (deve falhar com erro 403).
10. Exclua a avaliação enviando o nome de usuário correto.

6. Dicas para a Implementação

- No **src/server.js**, importe todos os Models que serão usados:

```
``javascript import Imovel from "../models/imovel.js"; import Reserva from
"./models/reserva.js"; import Avaliacao from "../models/avaliacao.js"; ``
```

- Na rota de criar reserva, busque o imóvel antes de calcular o valor:

```
``javascript const imovel = await Imovel.findById(imovelId); ``
```

- Para calcular o valor total da reserva:

```
``javascript const valorTotal = quantidadeNoites * imovel.precoNoite; ``
```

- No modelo de avaliações, valide a nota com **min: 1** e **max: 5** no próprio Schema do Mongoose.

- Use o **try/catch** para capturar as validações disparadas pelo Schema do Mongoose na hora de salvar.
-

LionsDev • Professor Nicolas Cardoso Motta
Exercício Prático de Mongoose e MongoDB - Módulo 08